

2026 암호분석경진대회

7번 문제

복원한 소수 p , q 와 복호화된 평문 m 의 바이트 문자열은 다음과 같다.

p	0xffa360d46885c534d186538179633fafc2c0548a2e24a2c1c878569f522939e38ff75ead1bcd442a834974a52e3ac66c1bc2ee00d63e2de4c436d2ca740a624699e1a1af94045c63261323cb723a7ba1b3c00bbbb6a4e534c11469e73ddbb2e50b0bc2461fcbac0726360c2c0ac9450a9a892cbf1d98ceee48827591ccc593c9
q	0xe557fa8670389cb60c84416a65742a74fd11ed33b1631f787e92b90887b5391dacba00a386911bf8a8fbd57430b9b26e455329405ffe289e20616fe3b5562ea9b533f8f8db94bb8dcd280a6af056108e176008d3655428ad0ac6396318ba0f6efe496eac3f8585675bfed67081e0c518be5685e4daf7060abelc58b73cc5f1e9
평문 m	FLAG{dlrty_blt_l34k_c0pp3rsm1th_m33ts_str4t3gy}

1. 문제 분석

문제에서 주어진 MASK 값은 다음과 같은 값으로,

MASK	ffffffffffffff ffffffff0fffff ffffffffffffffff c0000000000000fffe 0000000000000000 000003ffe0000000 00000000000000ffff ffffffffffffffff ffffffffffffff fffffff000000000 000003ffe0000000 000000000000001ff ffffffffffffff fffffffc3ffff ffffffffffffffff ffffffffffffffff
p & MASK	ffa360d46885c534 d186538170633faf c2c0548a2e24a2c1 c00000000000039e2 0000000000000000 000000a520000000 000000000003e2de4 c436d2ca740a6246 99e1a1af94045c63 261323c000000000 000003bba0000000 00000000000000e5 0b0bc2461fcbac07 26360c2c0809450a 9a892cbf1d98ceee 48827591ccc593c9

해당 MASK로 마스킹된 p & MASK 값을 복원하기 위해서는 다음과 같은 모듈러 방정식을 풀어야 한다.

$$f(x_0, x_1, \dots, x_6) = p_{leak} + 2^{150}x_0 + 2^{265}x_1 + 2^{362}x_2 + 2^{600}x_3 + 2^{682}x_4 + 2^{784}x_5 + 2^{920}x_6 = 0 \pmod{p}$$

$x_0, x_1, \dots, x_6$ 의 값의 범위는 다음과 같다.

x_0	4-bit : $0 \leq x_0 \leq 2^4 - 1$
x_1	84-bit : $0 \leq x_1 \leq 2^{84} - 1$
x_2	58-bit : $0 \leq x_2 \leq 2^{58} - 1$
x_3	69-bit : $0 \leq x_3 \leq 2^{69} - 1$
x_4	87-bit : $0 \leq x_4 \leq 2^{87} - 1$
x_5	46-bit : $0 \leq x_5 \leq 2^{46} - 1$
x_6	4-bit : $0 \leq x_6 \leq 2^4 - 1$

Coppersmith Method를 통한 다변수 선형 모듈러 방정식의 풀이 가능 여부는 다음 Herrmann-May Theorem[1]을 통해 확인할 수 있다.

Theorem. $\epsilon > 0$, N 은 충분히 큰 합성수이고 $p \geq N^\beta$ 인 약수 p 를 가진다. $f(x_1, \dots, x_n)$ 이 monic linear polynomial일 때, 다음을 만족하면 모든 해 $|y_i| \leq N^{r_i}$ 를 찾을 수 있다.

$$\sum_1^n r_i \leq 1 - (1 - \beta)^{\frac{n+1}{n}} - (n+1)(1 - (1 - \beta)^{\frac{1}{n}})(1 - \beta) - \epsilon$$

N 을 2048-bit 값, β 을 $\frac{1}{2}$ 이라 가정했을 때, 위 부등식의 LHS 부분은 $\sum_1^7 r_i = \frac{352}{2048} = 0.171875$, 우변 RHS 부분

의 값은 0.170033으로 위 부등식을 만족한다. 이는 문제의 방정식을 이론적으로 풀 수 있음을 시사한다. 그러나 margin 값이 $0.171875 - 0.170033 = 0.001842$ 로 작으며, 논문 [1]에서 Coppersmith Method을 위한 격자 파라미터 m 을 다음과 같이 잡음을 고려할 때

$$m = \left\lceil \frac{n(\frac{1}{\pi}(1-\beta)^{-0.278465} - \beta \ln(1-\beta))}{\epsilon} \right\rceil$$

문제의 식을 풀기 위한 m 은 최소 2785 이상으로 선택된다. 또한 구성되는 격자의 차원은 $\binom{m+n}{n}$ 으로 구성되어 LLL의 계산량이 비현실적으로 많아지게 된다. 이 때문에 7변수 방정식을 다음과 같은 방법을 통해 2변수 방정식으로 줄이는 과정을 거쳤다.

1. x_0, x_6 는 각각 4-bit 값이므로 도합 8-bit의 brute force 방식을 통해 추측한다.
2. 미지수의 개수를 줄이기 위하여 마스킹 된 블록의 개수를 2개로 줄인다. 방법 1.을 적용했을 때 마스킹 된 지점은 총 다섯 지점으로, 이때 x_1 과 x_2 가 포함된 p & MASK의 265~420번째 비트 구간과 x_3, x_4, x_5 이 포함된 600~830번째 비트 구간을 통째로 모른다고 설정하여, 미지수의 개수를 2개로 줄인다.

방법 2.를 적용함으로써 모르는 비트의 수는 41-bit가 추가된다. 그러나 41-bit가 추가되어도 Herrmann-May bound를 초과하지 않으며, 오히려 격자의 차원과 격자 파라미터 m 값이 크게 감소하여, 비로소 Coppersmith Method 연산이 가능해진다. 실제로 다음과 같은 방정식을 고려하였을 때,

$$f(x,y) = p_{leak}' + 2^{265}x + 2^{600}y = 0 \bmod p$$

x	155-bit : $0 \leq x \leq 2^{155} - 1$
y	230-bit : $0 \leq y \leq 2^{230} - 1$

Herrmann-May bound는 다음과 같다.

$$(\text{LHS}) \sum_1^2 r_i = \frac{385}{2048} = 0.187988$$

$$(\text{RHS}) = 0.207107$$

$$\text{margin} = 0.207107 - 0.187988 = 0.019118$$

이 경우 격자 파라미터 m 은 77 정도로 잡으면 충분하다. 따라서 기존 x_0, x_6 부분은 전수조사로 값을 추측해가며 $f(x, y) = 0 \bmod p$ 을 Coppersmith Method로 푸는 방식으로 RSA의 비밀 값 p 을 복원할 수 있다.

2. 코드 구현

공격 코드는 sage로 구현하였으며, Coppersmith Method를 적용하기 위한 라이브러리로 cuso, msolve, flatter 모듈을 사용하였다[2]. x_0 , x_6 값은 0부터 255의 전수조사를 통해 각각 0x9, 0xb 임을 확인하였다. 코드 실행은 결과는 다음과 같다.

[illegible]

See <https://github.com/sagemath/sage/issues/38942> for details.

```
if isinstance(key, MPolynomial) and not key.is_generator():
```

[16:47:04] INFO: Recentering variable y_0 from $-2^{154.0} < y_0 < 2^{154.0}$ to $-2^{154.0} < y_{0_c} < 2^{154.0}$

[16:47:04] INFO: Recentering variable y_1 from $-2^{229.0} < y_1 < 2^{229.0}$ to $-2^{229.0} < y_{1_c} < 2^{229.0}$

[16:47:04] INFO: Attempting the automated multivariate Coppersmith method with 2 variable(s)

[16:47:04] INFO: Multivariate Coppersmith problem has 2 modular relation(s) and 0 integer relation(s)

[16:47:05] INFO: Recentering variable y_0 from $-2^{154.0} < y_0 < 2^{154.0}$ to $-2^{154.0} < y_{0_c} < 2^{154.0}$

[16:47:05] INFO: Recentering variable y_1 from $-2^{229.0} < y_1 < 2^{229.0}$ to $-2^{229.0} < y_{1_c} < 2^{229.0}$

[16:47:05] INFO: Recovered 1 modular ideal(s)

[16:47:05] INFO: Generated ideal of multiplicity 1 with 1023.997960-bit modulus

[16:47:05] INFO: Computed 21 shift polynomials with monomials smaller than the modulus

[16:47:05] INFO: Generated ideal of multiplicity 2 with 2047.995919-bit modulus

[16:47:05] INFO: Computed 70 shift polynomials with monomials smaller than the modulus

[16:47:05] INFO: Generated ideal of multiplicity 3 with 3071.993879-bit modulus

[16:47:05] INFO: Computed 151 shift polynomials with monomials smaller than the modulus

[16:47:05] INFO: Generated ideal of multiplicity 4 with 4095.991838-bit modulus

[16:47:08] INFO: Computed 261 shift polynomials with monomials smaller than the modulus

[16:47:08] INFO: Generated ideal of multiplicity 5 with 5119.989798-bit modulus

[16:47:22] INFO: Computed 398 shift polynomials with monomials smaller than the modulus

[16:47:22] INFO: Generated ideal of multiplicity 6 with 6143.987758-bit modulus

[16:47:33] INFO: Graph optimization found a subset of 275 shift relations (out of 384)

[16:47:33] INFO: Built a dual lattice basis of rank 275 and dimension 275

[16:51:34] INFO: Reduced lattice basis using flatter

[16:51:35] INFO: Found 218 integer relations in the lattice

/home/ubuntu/anaconda3/envs/sage/lib/python3.11/site-packages/cuso/data/bounds/bound_set.py:

133: DeprecationWarning: is_generator is deprecated. Please use is_gen instead.

See <https://github.com/sagemath/sage/issues/38942> for details.

```
if isinstance(key, MPolynomial) and not key.is_generator():
```

[16:51:39] INFO: Found 1 possible (partial) root(s)

[16:51:40] INFO: Found partial solution

[16:51:40] INFO: $y_0 = 9837175923679103295934196095171519333829373401$

[16:51:40] INFO: $y_1 = -634365610644523845525827760520279723068495709572242168029738148429610$

[16:51:40] INFO: $p = 1795152462558155920493827008124718934441905020514069937404373579784819988461259284327131383$
 $9947046384066731487246202837751799204492332258806297260808525498728698426736924616545989026$
 $8602041631816389172453661734209705876325960948672128367147143904100252385980226355574027573$
 $688585975211787264944330620690338761$

[16:51:40] INFO: cuso elapsed=275.3s

[16:51:40] INFO: $\text{root}=\{y_0: 9837175923679103295934196095171519333829373401, y_1:$
 $-634365610644523845525827760520279723068495709572242168029738148429610, 'p':$
 $1795152462558155920493827008124718934441905020514069937404373579784819988461259284327131383$
 $9947046384066731487246202837751799204492332258806297260808525498728698426736924616545989026$
 $8602041631816389172453661734209705876325960948672128367147143904100252385980226355574027573$
 $688585975211787264944330620690338761\}$

[16:51:40] INFO: SUCCESS

```

[16:51:40] INFO: p =
0xffa360d46885c534d186538179633fafc2c0548a2e24a2c1c878569f522939e38ff75ead1bcd442a834974a52e3
ac66c1bc2ee00d63e2de4c436d2ca740a624699e1a1af94045c63261323cb723a7ba1b3c00bbbb6a4e534c11469
e73ddbb2e50b0bc2461fcbac0726360c2c0ac9450a9a892cbf1d98ceee48827591ccc593c9
[16:51:40] INFO: q =
0xe557fa8670389cb60c84416a65742a74fd11ed33b1631f787e92b90887b5391dacba00a386911bf8a8fbd57430
b9b26e455329405ffe289e20616fe3b5562ea9b533f8f8db94bb8dcd280a6af056108e176008d3655428ad0ac6396
318ba0f6efe496eac3f8585675bfed67081e0c518be5685e4daf7060abe1c58b73cc5f1e9
[16:51:40] INFO: plaintext hex =
464c41477b64317274795f6231745f6c33346b5f633070703372736d3174685f6d333374735f73747234743367797
d
[16:51:40] INFO: plaintext bytes = b'FLAG{dlrty_b1t_l34k_c0pp3rsm1th_m33ts_str4t3gy}'
[16:51:40] INFO: saved = test.txt
[16:51:40] INFO: winning guess=0x9b (155)

```

3. 참고 문헌

- [1] Herrmann, M., & May, A. (2008, December). Solving linear equations modulo divisors: On factoring given any bits. In International Conference on the Theory and Application of Cryptology and Information Security (pp. 406-424). Berlin, Heidelberg: Springer Berlin Heidelberg.
- [2] <https://github.com/keeganryan/cuso>